Money Laundering Detection with Machine Learning

Course project, Information Systems Security

Nguyễn Thế Vĩnh
26848266@grad.uit.edu.vn
University of Information Technology, VNU-HCM
Ho Chi Minh City, Vietnam

Hoàng Đức Chung
26848014@grad.uit.edu.vn
University of Information Technology, VNU-HCM
Ho Chi Minh City, Vietnam

Nguyễn Hải Đăng
26848027@grad.uit.edu.vn
University of Information Technology, VNU-HCM
Ho Chi Minh City, Vietnam

ABSTRACT

Machine learning for money laundering detection faces two obstacles: the positive class is extremely rare, and the signal lies in behavioral context rather than in individual transactions. We compare two ways of improving a detector within a single evaluation protocol on the 5,078,345 transactions of the synthetic IBM AML HI-Small dataset: *data augmentation* of the positive class with random oversampling, SMOTENC, CTGAN, and a Large Language Model (LLM), and *better representations* through temporal behavioral features. We build 63 features describing account history, windowed activity, and counterparty relationships, computed strictly from transactions that occur before the prediction time, and keep an XGBoost classifier fixed. Adding the feature groups step by step raises Average Precision (AP) from 0.171 to 0.534 and 0.684; with an alert budget of the 1,000 highest-scoring transactions, the full configuration reaches 91.50% precision and 50.89% recall. In contrast, no augmentation method clearly improves over no augmentation; the two LLM variants lower AP from 0.670 to 0.609 and 0.597, and the samples generated by CTGAN or the LLM are almost perfectly separable from real data (discriminator AUC $\geq$ 0.983). A further analysis reveals a strong distribution shift starting on September 11, 2022; on the part of the test set before that date, the benefit of behavioral features persists. For transaction data, feature quality and the temporal correctness of the pipeline matter more than adding synthetic data.

KEYWORDS

anti-money laundering, transaction monitoring, XGBoost, temporal features, feature engineering, data augmentation, large language models, data leakage

1 INTRODUCTION

Money laundering disguises the illicit origin of funds through chains of transactions. Financial institutions must monitor transactions and report suspicious activity under the FATF Recommendations [12] and, in Vietnam, under the 2022 Law on Anti-Money Laundering [14]. From an information security perspective, a transaction monitoring system plays a role similar to an intrusion detection system: it must rank a very large stream of events of which only a small fraction is malicious, and every false positive costs investigator time.

Machine learning for money laundering detection faces three difficulties. First, the positive class is extremely rare: in the IBM AML HI-Small dataset [3] only about 0.1% of transactions are labeled as laundering. Second, a single transaction carries almost no signal; the evidence lies in its *context*, such as account history, transaction velocity, and relationships between accounts. Third, the data is ordered in time, so it is easy to let the model "see the future" during feature engineering (temporal leakage), which produces over-optimistic results.

There are two common ways to improve a detector. The first is *data augmentation* of the positive class through oversampling, SMOTE [6], generative models such as CTGAN [18], or, more recently, LLMs [5, 15]. The second is *feature engineering* that describes the temporal behavior of accounts and the relationships between them [4, 8]. We compare both under one protocol: the same temporal split, the same XGBoost classifier [7] with fixed hyperparameters, the same threshold selection on the validation set, and three seeds. Our contributions are:

- A set of 63 behavioral features computed *strictly in time*: only transactions with an earlier timestamp are used, transactions within the same minute share one history snapshot, and money totals are kept separate per currency.
- A controlled ablation over five predefined configurations, showing that AP grows from 0.171 to 0.684 and that every added group improves AP for all three seeds.
- An evaluation of four data augmentation methods, including two LLM-based generators, together with quality diagnostics of the synthetic data. None clearly improves the detector; CTGAN and LLM samples are almost perfectly separable from real data.
- An analysis of evaluation validity covering the time policy, a distribution shift starting on September 11, 2022, and test-set reuse, together with a reproducible verification procedure.

2 RELATED WORK

Datasets and benchmarks. Since real bank data cannot be published, academic work relies on public data such as the Elliptic Bitcoin transaction graph [17] or on simulated data. Altman et al. [3] released the AMLworld agent-based generator and six fully labeled synthetic banking datasets [2]; HI-Small is the smallest one with the higher illicit ratio.

Graph features and GNNs. The Graph Feature Preprocessor (GFP) [4], part of Snap ML [9], extracts fan-in/fan-out, scatter-gather, and cycle patterns together with windowed vertex statistics in real time and feeds them to gradient boosting models; on HI-Small this reaches a minority-class F1 of about 0.63–0.65 and outperforms standard Graph Neural Networks (GNNs). Multi-GNN [11] extends GNNs to directed multigraphs. Deprez et al. [8] survey 97 studies and find that network information improves

Table 1: Temporal split of the data.

Set	Transactions	Positives	Rate	Time range
Train	3,046,861	2,297	0.075%	Sep 1 – Sep 6 13:35
Validation	1,015,602	1,082	0.107%	Sep 6 13:36 – Sep 8 16:11
Test	1,015,882	1,798	0.177%	Sep 8 16:12 – Sep 18

predictive performance. Recent work focuses on fast pattern mining [19] or quasi-temporal graph representations [16]. In contrast, Effendi and Chattopadhyay [10] observe that XGBoost may not need graph features in some data settings. We do not reproduce GFP; we focus on local behavioral and relationship statistics that are directly interpretable.

Data augmentation and LLMs. SMOTE [6] interpolates between minority samples; CTGAN [18] learns tabular distributions with a conditional GAN. LLMs have been used to generate tabular data [5], combined with data curation in low-data regimes [15], or to propose features automatically [1, 13]. We evaluate LLM-based generation of laundering transactions in feature space for an extremely imbalanced problem.

3 DATA AND PROBLEM

Dataset. We use the file `HI-Small_Trans.csv` of IBM AML [2], which contains 5,078,345 transactions between September 1 and 18, 2022, of which 5,177 (0.102%) are labeled as laundering. Each transaction has a timestamp (minute resolution), sender and receiver bank and account, amount and currency on both sides, payment format, and label. The file is checked with a SHA-256 hash between the local and Kaggle environments. Accounts are identified by the bank–account pair; identifiers are only used to look up history and are never model inputs.

Task. Each transaction is a sample. The model outputs a suspicion score for ranking; binary decisions use a threshold chosen on the validation set. Scores are not calibrated as risk probabilities.

Temporal split. Transactions are sorted by timestamp and split approximately 60/20/20; each cut is moved to the start of its minute so that no minute is split across two sets (Table 1). The data is not shuffled, and validation and test labels are never used for feature engineering or training.

4 METHOD

4.1 Time policy

Let t be the timestamp of the transaction to be scored. The history that may be observed is

$$H(t) = \{\, j : t_j < t \,\}. \tag{1}$$

Because timestamps only have minute resolution and the true order within a minute is unknown, all transactions with the same timestamp share one history snapshot; the history is updated only after the whole group has been queried. Features therefore do not depend on the row order of the file. A self-transfer creates one outgoing and one incoming event, and the outgoing event of a transaction never becomes history for its own receiving side. A time window of length W is defined as $[t - W, t)$. Unlabeled transactions that already occurred in the validation and test periods may contribute history to later transactions, which corresponds to observing the transaction stream over time.

4.2 Behavioral features

The feature set has four groups and 63 columns in total:

- **Transaction (11):** log amount paid and received, hour, weekday, payment format, both currencies, same bank, self-transfer, currency conversion, and round amount.
- **History (17):** for both sender and receiver: number of previous transactions, number of outgoing and incoming transfers, mean log amount sent and received, time since the previous transaction, and number of transactions on the same day in total and per direction; plus the number of previous transactions of the sender→receiver pair.
- **Activity (20):** for 1-hour and 24-hour windows, the count and log total amount per direction for each account, and the deviation of the current log amount from the historical mean log amount in the same direction. Totals and deviations are kept separate per account *and currency*; different currencies are never added without an exchange rate. Sliding sums use Neumaier compensated summation so that no residual error remains when amounts of very different magnitude leave the window.
- **Relationship (15):** number of distinct counterparties per direction and window, previous transactions of the reverse pair, age and recency of the sender→receiver relationship, pair counts within each window, and the 24-hour counterparty share (transactions A→B divided by A's outgoing transfers).

Without history, counts and totals are 0, while means and time gaps are left as NaN for XGBoost to handle. Five configurations were fixed before running: F0 = Transaction; F1 = F0 + History; F2 = F1 + Activity; F3 = F1 + Relationship; F4 = F1 + Activity + Relationship. The difference F1−F0 measures the effect of history; F2−F1 and F3−F1 measure each group's benefit once history is present; F4−F2 and F4−F3 measure what the remaining group adds. Features are computed with linear per-account scans accelerated by Numba.

4.3 Data augmentation

Augmentation only adds synthetic laundering transactions to the training set; validation and test always contain real data. The number of synthetic samples is 0.5 times the number of positive training samples (1,148 per run). We compare: **ER** random oversampling of positive samples; **E1** SMOTENC on standardized features, with categorical features taken as the mode of the nearest neighbors; **E2** CTGAN trained for 300 epochs on positive samples only; and **E3a/E3b** a commercial LLM accessed through the OpenAI API that generates laundering transactions as JSON from real examples, column statistics, and logical constraints between columns, where E3b also describes the AMLworld laundering typologies in the prompt. LLM outputs are validated, deduplicated, and filtered for verbatim copies of real rows; API cost is capped by a budget ledger. Synthetic data quality is measured by the mean per-column

Kolmogorov–Smirnov (KS) statistic against real positive samples and by the AUC of an XGBoost discriminator that separates real from synthetic rows (*discriminator AUC*; 0.5 means indistinguishable).

The augmentation experiment was run before the feature study, on a 28-feature pipeline corresponding to F1 that processed transactions within the same minute in the stable row order of the file. Because the protocol differs, we only compare augmentation methods with each other, not their absolute scores with F0–F4.

4.4 Model and evaluation

All configurations use XGBoost with at most 1,000 trees, learning rate 0.05, max depth 8, subsample and colsample_bytree 0.8, minimum child weight 1, the `hist` tree method on GPU, and early stopping after 100 rounds on validation AUC-PR. There is no per-configuration hyperparameter tuning and no class resampling. The F1-maximizing threshold is selected on the validation set of each run and applied unchanged to the test set.

The primary metric is Average Precision (AP), computed with scikit-learn. We also report F1 at the threshold and Precision@1000/Recall@1000: the fraction of laundering transactions among the 1,000 highest-scoring test transactions, and the fraction of all laundering test transactions that appear in that list. Accuracy is not used because positives are very rare. Each configuration is run with three seeds (0, 1, 2); tables show the mean ± sample standard deviation (SD), and comparisons use paired differences per seed. This SD reflects training variability on the same test set, not a confidence interval over new transaction populations.

4.5 Verification and reproducibility

The pipeline has 43 regression tests covering time and window boundaries, self-transfers, currencies, invariance to reordering rows with the same timestamp, independence from labels and future data, and agreement with a direct computation on small samples. After the runs, the 30 prediction files of the 15 runs are rechecked for labels, ordering, AP, threshold metrics, and validation-based threshold selection. A manifest records the hashes of the data and source code, library versions, column lists, and split information. Runs were executed in a private Kaggle notebook with two Tesla T4 GPUs, NumPy 2.2.6, pandas 2.3.3, scikit-learn 1.7.2, XGBoost 3.1.1, and Numba 0.61.2.

5 RESULTS

5.1 Feature group ablation

Table 2 shows that transaction attributes alone (F0) yield an AP of 0.171. Adding account history (F1) gives the largest jump, to 0.534. Once history is present, the Activity group (F2, 0.662) contributes more than the Relationship group (F3, 0.610), and combining both (F4) reaches the highest AP, 0.684. The ranking of configurations is the same on validation and test. Table 3 shows that every addition improves AP for all three seeds, including F4 over F2: the Relationship group still adds information when the Activity group is present. Three seeds only describe the consistency of these runs and are not sufficient to claim statistical significance.

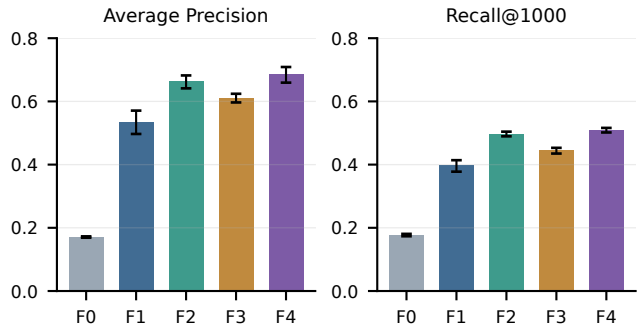


Figure 1: AP and Recall@1000 of the five feature configurations; error bars show the SD over three seeds on the same test set.

5.2 Alert budget

The test set contains 1,798 laundering transactions. Among the 1,000 highest-scoring alerts, F1 finds about 712 laundering transactions on average and F4 finds 915; false positives drop accordingly while the investigators' workload stays at 1,000 alerts. However, a recall of 0.509 also means that almost half of the laundering transactions fall outside the list. A budget of 1,000 alerts for the whole test set is an experimental assumption; a real system may need daily limits or limits based on investigation capacity.

5.3 Feature importance

Table 4 shows that the most important features combine the current transaction with account context: the sender's number of previous incoming transfers, the receiver's time since its previous transaction, the deviation of the amount from same-currency history, the payment format, and the number of previous transactions of the account pair. This matches domain intuition: an account that has just received funds and forwards an amount that is unusual for its history is a typical sign of a money mule. However, tree gain importance is affected by correlations between columns, so it should not be read as a causal effect and does not replace the group ablation above.

5.4 Data augmentation

Table 5 shows that no augmentation method clearly improves over no augmentation (AP 0.670). SMOTENC reaches AP 0.673, a paired difference of only 0.0026 that lies within the variation between seeds. Random oversampling slightly lowers AP (ΔAP = −0.0077), CTGAN lowers it by −0.0221, and the two LLM variants lower it the most (−0.0611 and −0.0732).

The discriminator AUC explains this pattern. Repeated samples are indistinguishable from real ones (0.502), SMOTENC reaches 0.647, whereas CTGAN (0.999) and the LLM (0.983 and 0.996) are almost perfectly separable. Synthetic samples still carry a positive label, so the model learns regions that do not exist in real data. With the LLM, precision at the threshold increases but recall drops sharply: the model becomes more "conservative" and misses more laundering transactions. Notably, adding domain knowledge about laundering typologies to the prompt (E3b) does not help and makes

Table 2: Feature group ablation (mean ± SD over three seeds). AP is reported on the test and validation sets; the other metrics are on the test set.

Configuration	Features	AP (test)	AP (validation)	F1	Precision@1000	Recall@1000
F0 Transaction	11	0.171 ± 0.002	0.064 ± 0.001	0.246 ± 0.016	0.318 ± 0.007	0.177 ± 0.004
F1 +History	28	0.534 ± 0.037	0.477 ± 0.026	0.534 ± 0.028	0.712 ± 0.033	0.396 ± 0.018
F2 +Activity	48	0.662 ± 0.020	0.583 ± 0.014	0.664 ± 0.016	0.893 ± 0.013	0.497 ± 0.007
F3 +Relationship	43	0.610 ± 0.014	0.521 ± 0.026	0.589 ± 0.026	0.798 ± 0.016	0.444 ± 0.009
F4 Combined	63	**0.684 ± 0.025**	**0.592 ± 0.023**	**0.685 ± 0.026**	**0.915 ± 0.013**	**0.509 ± 0.007**

Table 3: Paired AP differences on the test set (same seed).

Comparison	Mean ΔAP	SD	Seeds improved
F1 − F0	+0.3631	0.0371	3/3
F2 − F1	+0.1278	0.0165	3/3
F3 − F1	+0.0765	0.0233	3/3
F4 − F1	+0.1502	0.0122	3/3
F4 − F2	+0.0224	0.0043	3/3
F4 − F3	+0.0737	0.0112	3/3

Table 4: Ten features with the highest mean total-gain share in F4 (three seeds).

#	Feature	Group	Gain (%)
1	Sender's number of previous incoming transfers	History	7.74
2	Receiver's time since its previous transaction	History	7.05
3	Amount deviation from sender's outgoing history (same currency)	Activity	6.35
4	Payment format	Transaction	5.27
5	Hour of day	Transaction	4.77
6	Previous transactions of the sender→receiver pair	History	4.64
7	Amount deviation from sender's incoming history	Activity	4.29
8	Amount deviation from receiver's incoming history	Activity	3.71
9	Log of amount received by sender in 24 hours	Activity	3.70
10	Sender's mean log amount received	History	3.51

results less stable across seeds (large F1 SD). In transaction data, each laundering transaction is part of a chain with temporal and relational structure; generating independent rows in feature space struggles to reproduce these constraints. This is the practical reason why we shifted our focus to feature engineering.

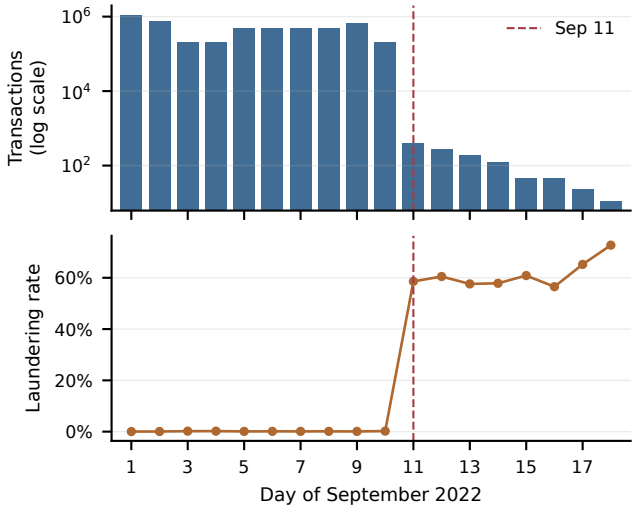


Figure 2: Daily transaction count (log scale) and positive rate in HI-Small; the dashed line marks September 11.

5.5 Temporal distribution shift

Daily statistics show that from September 11, 2022 the transaction volume drops sharply while the positive rate jumps (Figure 2): September 10 has 208,325 transactions and 442 positives, whereas September 11 has only 396 transactions but 232 positives. Within the test set, the part before September 11 contains 1,014,774 transactions with 1,143 positives (0.113%), while the part from September 11 contains only 1,108 transactions but 655 positives (59.1%). The cause has not been verified from the generation mechanism. Because this is a post-hoc finding, we neither remove this part from the main evaluation nor retrain; instead, we split the fixed predictions into two parts for diagnosis (Table 6).

Before September 11, F4 still outperforms F1 (AP 0.528 versus 0.333), so the benefit of the features does not come only from the anomalous tail. In the tail, transaction attributes alone (F0) already reach AP 0.973 and F4 does not outperform F0. Since AP depends on the positive rate, we only compare configurations within the same part and do not compare AP across parts to conclude which part is easier.

Table 5: Training-set augmentation (earlier pipeline protocol, 28 features; three seeds). KS and discriminator AUC measure the distance between synthetic and real positive samples; a discriminator AUC closer to 0.5 means more realistic synthetic data.

Method	AP	F1	Precision@1000	Mean KS	Discriminator AUC
E0 No augmentation	0.670 ± 0.008	0.673 ± 0.010	0.902	–	–
ER Random oversampling	0.662 ± 0.010	0.664 ± 0.003	0.894	0.019	0.502
E1 SMOTENC	0.673 ± 0.014	0.674 ± 0.009	0.899	0.047	0.647
E2 CTGAN	0.648 ± 0.014	0.625 ± 0.016	0.883	0.221	0.999
E3a LLM, basic prompt	0.609 ± 0.015	0.588 ± 0.018	0.854	0.126	0.983
E3b LLM, AML-aware prompt	0.597 ± 0.051	0.493 ± 0.181	0.838	0.139	0.996

Table 6: AP on the two parts of the test set, split at September 11, 2022 (post-hoc diagnostic, same models).

Configuration	Before Sep 11	From Sep 11
F0	0.071 ± 0.000	0.973 ± 0.000
F1	0.333 ± 0.042	0.962 ± 0.004
F2	0.506 ± 0.022	0.958 ± 0.005
F3	0.413 ± 0.022	0.962 ± 0.004
F4	0.528 ± 0.028	0.961 ± 0.005

5.6 Computational cost

On Kaggle, building the full feature table (including data loading and JIT compilation) takes about 35.0 seconds, and the feature table occupies about 1.33 GB. Training the 15 models takes 620.2 seconds; the whole run, including evaluation and file output, takes 733.3 seconds. These numbers describe one run in one environment; we have not measured per-transaction feature update latency in an online service.

6 DISCUSSION

Implications for information security. A transaction monitoring system is a defensive component; its quality is measured by the number of laundering transactions found within the alert budget that investigators can handle. Our results show that feature engineering determines effectiveness far more than adding synthetic data, and behavioral features have the additional advantage of being explainable in business terms, which matters when alerts must be reviewed by investigators. On the other hand, a behavior-based representation also creates an attack surface: launderers can deliberately split transactions (structuring), spread them over time, or add money mules to distort windowed and relational statistics (evasion attacks). In deployment, the system also needs access control for the data and protection of API keys and alert results; our published code contains no API keys, raw data, or per-transaction predictions.

Temporal leakage as a methodological vulnerability. A high score is meaningless if features use future information, labels of the evaluation set, or an arbitrary row order of the file. The earlier augmentation pipeline used 28 features corresponding to F1 but processed same-minute transactions in row order, and reached AP 0.670 without augmentation, whereas F1 under the strict time policy reaches only 0.534. The two pipelines also differ in some implementation details, so this is not a controlled comparison. Still, the large gap suggests that information about same-minute transactions and their order in the file can inflate scores considerably. When comparing feature sets or comparing with other work, the time policy must therefore be fixed and reported, just like the model and the temporal split. For the same reason, the F1 of F4 (0.685) should not be compared directly with published HI-Small results [4], which use different threshold selection and time handling.

Limitations. (i) The data is simulated; labels come from the generator, with no KYC, balances, or verification of the source of funds. (ii) The HI-Small test set was examined in the earlier augmentation experiment, so although the five configurations were predefined, the results remain exploratory. (iii) Three seeds only measure training variability and do not establish statistical significance on independent data. (iv) Hyperparameters are fixed, so the results show the effect of features under one training budget rather than maximum performance; some runs reach or approach the 1,000-tree limit. (v) The mean log amounts in the History group still mix currencies. (vi) The Relationship group only contains local statistics and does not detect cycles or multi-hop chains. (vii) The augmentation experiment uses a different protocol. (viii) The tail of the test set has an anomalous distribution. (ix) The results have not been validated on other AMLworld datasets or on real bank data.

7 CONCLUSION

On IBM AML HI-Small, behavioral feature engineering yields large and consistent gains, whereas synthetic data does not. With a fixed XGBoost model, adding history, activity, and relationship features raises AP from 0.171 to 0.684; among the 1,000 highest-scoring alerts, the full configuration reaches 91.50% precision and 50.89% recall. The Activity group contributes more than the Relationship group when added alone, and the Relationship group still helps when combined. Random oversampling, SMOTENC, CTGAN, and both LLM variants fail to improve clearly; samples generated by CTGAN and the LLM are almost perfectly separable from real data. Results on the test period before September 11 confirm the benefit of the features, while the anomalous tail is a reminder that conclusions may not hold uniformly across all periods of the data.

Future work includes comparing with multi-hop graph pattern features such as GFP under the same time policy, analyzing results per laundering typology, evaluating robustness against evasion strategies such as structuring or money mules, and validating on other AMLworld datasets with a completely new held-out test set.

Reproducibility. Source code, configurations, tests, and per-run results are available at https://github.com/Chung10102004/atbmtt. Raw data and per-transaction predictions are not included in the repository.

REFERENCES

[1] Nikhil Abhyankar, Parshin Shojaee, and Chandan K. Reddy. 2025. LLM-FE: Automated Feature Engineering for Tabular Data with LLMs as Evolutionary Optimizers. arXiv:2503.14434

[2] Erik Altman. 2023. IBM Transactions for Anti Money Laundering (AML). https://www.kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering-aml. File HI-Small_Trans.csv.

[3] Erik Altman, Jovan Blanuša, Luc von Niederäusern, Béni Egressy, Andreea Anghel, and Kubilay Atasu. 2023. Realistic Synthetic Financial Transactions for Anti-Money Laundering Models. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

[4] Jovan Blanuša, Maximo Cravero Baraja, Andreea Anghel, Luc von Niederäusern, Erik Altman, Haris Pozidis, and Kubilay Atasu. 2024. Graph Feature Preprocessor: Real-time Subgraph-based Feature Extraction for Financial Crime Detection. In *Proceedings of the 5th ACM International Conference on AI in Finance (ICAIF)*. https://doi.org/10.1145/3677052.3698674 arXiv:2402.08593.

[5] Vadim Borisov, Kathrin Seßler, Tobias Leemann, Martin Pawelczyk, and Gjergji Kasneci. 2023. Language Models are Realistic Tabular Data Generators. In *International Conference on Learning Representations (ICLR)*.

[6] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. 2002. SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research* 16 (2002), 321–357.

[7] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 785–794. https://doi.org/10.1145/2939672.2939785

[8] Bruno Deprez, Toon Vanderschueren, Bart Baesens, Tim Verdonck, and Wouter Verbeke. 2024. Network Analytics for Anti-Money Laundering – A Systematic Literature Review and Experimental Evaluation. *INFORMS Journal on Data Science* (2024). https://doi.org/10.1287/ijds.2024.0042 arXiv:2405.19383.

[9] Celestine Dünner, Thomas Parnell, Dimitrios Sarigiannis, Nikolas Ioannou, Andreea Anghel, Gummadi Ravi, Madhusudanan Kandasamy, and Haralampos Pozidis. 2018. Snap ML: A Hierarchical Framework for Machine Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[10] Fabrianne Effendi and Anupam Chattopadhyay. 2024. Privacy-Preserving Graph-Based Machine Learning with Fully Homomorphic Encryption for Collaborative Anti-Money Laundering. arXiv:2411.02926

[11] Béni Egressy, Luc von Niederäusern, Jovan Blanuša, Erik Altman, Roger Wattenhofer, and Kubilay Atasu. 2024. Provably Powerful Graph Neural Networks for Directed Multigraphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

[12] Financial Action Task Force. 2012. International Standards on Combating Money Laundering and the Financing of Terrorism & Proliferation – The FATF Recommendations. https://www.fatf-gafi.org. Updated periodically.

[13] Noah Hollmann, Samuel Müller, and Frank Hutter. 2023. Large Language Models for Automated Data Science: Introducing CAAFE for Context-Aware Automated Feature Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[14] National Assembly of Vietnam. 2022. Law on Anti-Money Laundering, No. 14/2022/QH15.

[15] Nabeel Seedat, Nicolas Huynh, Boris van Breugel, and Mihaela van der Schaar. 2024. Curated LLM: Synergy of LLMs and Data Curation for Tabular Augmentation in Low-Data Regimes. In *International Conference on Machine Learning (ICML)*.

[16] Haseeb Tariq and Marwan Hassani. 2026. Extracting Money Laundering Transactions from Quasi-Temporal Graph Representation. arXiv:2604.02899

[17] Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I. Weidele, Claudio Bellei, Tom Robinson, and Charles E. Leiserson. 2019. Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics. In *KDD Workshop on Anomaly Detection in Finance*.

[18] Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. 2019. Modeling Tabular Data using Conditional GAN. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[19] Haojie Ye, Arjun Laxman, Yichao Yuan, Krisztian Flautner, and Nishil Talati. 2026. BlazingAML: High-Throughput Anti-Money Laundering (AML) via Multi-Stage Graph Mining. arXiv:2604.12241